

Higher or lower

Overview

The first genre chapter. Students meet one small game three times - simple, intermediate, advanced - and the real subject is **how a game's code grows**: from one scene with a few fields, to a model (`Card` , `Deck` , `rules.ts`) kept apart from a view (`CardSprite` , `Button` , `HistoryRow`), with a scene in between. Along the way: a union-of-strings state machine that ignores input at the wrong moment, a Fisher-Yates shuffle (and why the popular `sort` trick is biased), a flip made of two tweens, a reusable button, and a risk-and-reward scoring system. There is little new Phaser here; there is a lot of design.

It is also the first chapter where students should *play critically*: why is the advanced version more fun than the simple one, when the underlying luck is the same?

Prerequisites

- Chapters 1-2: scenes, `init(data)` , interfaces, pointer input, the "scenes are reused" trap
- Chapter 3: a preload scene; Chapter 5: playing sounds; Chapter 6: local storage
- Chapter 7: tweens, tween chains, particles
- Java: `final` fields, getters, enums, `Predicate<T>` (for challenges 3 and 6)

Learning outcomes

Students can:

1. separate a game's model (plain TypeScript data and rules) from its view (Phaser game objects), and explain what each change touches
2. use unions of string literals for small sets of values (suits, guesses, outcomes, states)
3. implement a Fisher-Yates shuffle and explain why `sort(() => Math.random() - 0.5)` is biased

4. keep a game state, and ignore input while the game is busy
5. animate a card flip with a tween chain and change a sprite's frame (or texture) mid-animation
6. build a reusable `Container`-based button with hover, pressed and disabled states
7. design scoring that rewards risk (a pot, a multiplier, a cash-out choice) and persist records

Suggested session plan (2 x 2 hours)

Session 1 - simple and intermediate

Time	Activity
0:00 - 0:10	Play all three versions for two minutes each, in order. Ask: "which is most fun, and <i>why</i> ? The luck is identical." Collect answers on the board (risk, streak, reveal)
0:10 - 0:30	Slides 3-7: genre notes, the simple version - <code>Card</code> , frames, <code>judge()</code> , <code>GameState</code> . Walk through <code>GameScene.guess()</code>
0:30 - 0:45	Live demo: delete the state check at the top of <code>guess()</code> , rebuild, and mash H. Ask what went wrong before explaining
0:45 - 1:00	Challenge 1 (aces high). Then ask who changed <code>rank</code> instead of <code>value</code> - see Common problems
1:00 - 1:25	Slides 8-11: model and view, the deck, the shuffle bias (run the experiment - see Demos), <code>draw()</code> and <code>Card undefined</code>
1:25 - 1:40	Slides 12-14: the flip, buttons, the state machine. Live-code: change <code>Sine.easeIn</code> / <code>easeOut</code> to <code>Linear</code> , then change <code>scaleX</code> to <code>scale</code>
1:40 - 2:00	Challenges 2 and 3

Session 2 - advanced

Time	Activity
0:00 - 0:20	Slides 15-17: the pot, multipliers, cash out. Discussion: "with only a score, what is the best strategy?" (always guess the likely way - no decisions). "With a pot?"
0:20 - 0:35	Slide 18: dealing, sprites that swap jobs, the history row, records, polish
0:35 - 0:45	Polish exercise: turn off one piece of polish at a time (comment out the shake, the float text, the sparkles). Which one did the class miss most?
0:45 - 1:15	Challenge 4 (jokers wild)
1:15 - 2:00	Challenges 5 and 6. Finish with the computer playing (challenge 6) on the projector

Key points to stress

- **The model never imports Phaser.** Point out that `Card.ts`, `Deck.ts` and `rules.ts` could run in a plain Deno script (they can - see Demos). The next chapter (memory match) needs a deck too
- **The scene asks, then tells.** `judge()` returns an outcome; the scene decides what sound and text go with it. Rules never play sounds
- **One gate for all input.** H, L and both buttons all call `guess()`; the state check lives there once. If students add a new input (challenge 5's D, challenge 6's computer), it must go through the same gate
- **Order of the reveal.** `reveal()` runs *after* the flip completes. Play the result sound before the card is visible and the tension goes - worth demonstrating
- **The shuffle is a correctness issue, not style.** The chart in the chapter is real data from V8, the engine in Chrome and Deno
- `value` **versus** `rank`. Two names for what are equal today and different tomorrow (aces high)

Common problems and errors

What students see	Cause	Fix
Every card shows as the next rank up; the king of spades shows as a card back	frame formula missing <code>- 1</code> : <code>suit * 13 + rank</code>	<code>suit * 13 + (rank - 1)</code>
Aces high "works" but the ace of spades is a card back	challenge 1 done by changing <code>rank</code> to 14, so the frame is <code>suit * 13 + 13</code>	change <code>value</code> , not <code>rank</code>
Mashing H deals two cards and judges both against the same card to beat	no <code>state</code> check at the top of <code>guess()</code>	<code>if (this.state !== "waiting") return;</code>
Card shrinks to a dot instead of flipping	tweening <code>scale</code> instead of <code>scaleX</code>	<code>scaleX</code> only
<code>TypeError: this.showFace is not a function</code> at the first flip (in the browser console); the build was fine	<code>onComplete: function () { this.showFace(card); } - a function has its own <code>this</code></code>	arrow function: <code>onComplete: () => { ... }</code>
<code>Type 'Card undefined' is not assignable to type 'Card'.</code>	<code>draw()</code> returning <code>this.cards.pop()</code> directly	check for <code>undefined</code> first (and throw)
<code>Error: The deck is empty</code> (console, in <code>Deck.draw</code>)	drawing from an empty deck - e.g. a challenge that plays through the whole deck	check <code>isEmpty()</code> before drawing
<code>Property 'add' in type 'HistoryRow' is not assignable to the same property in base type 'Container'. (and This member must have an 'override' modifier because it overrides a member in the base class 'Container'.)</code>	naming a method <code>add</code> in a <code>Container</code> subclass	pick another name, e.g. <code>addCard</code>
<code>Argument of type '"down"' is not assignable to parameter of type 'Guess'.</code>	<code>this.guess("down")</code>	the union type doing its job - use <code>"lower"</code>

What students see	Cause	Fix
This comparison appears to be unintentional because the types 'Guess' and '"hihger"' have no overlap.	a typo in a string being compared	the union catches typos in comparisons too
Cannot assign to 'rank' because it is a read-only property.	trying to change a card (often while attempting aces high)	cards do not change; change <code>value</code> 's getter
Texture "cards" has no frame "joker" (console warning) and the ace of clubs appears	<code>setFrame(JOKER_KEY)</code> for a texture that is not in <code>cards.png</code>	<code>setTexture(JOKER_KEY)</code>
No sound until the first click	the browser's autoplay rule (Chapter 5)	expected; the title screen's PLAY click unlocks audio

Suggested demos and live-coding moments

- **The shuffle experiment.** In a scratch file, run in Deno:

```
const counts: Record<string, number> = {};
for (let i = 0; i < 600000; i++) {
  const order = ["A", "B", "C"].sort(() => Math.random() - 0.5).join("");
  counts[order] = (counts[order] ?? 0) + 1;
}
console.log(counts);
```

ABC and BAC come out about 150,000 times each; the other four about 75,000. Then swap in Fisher-Yates and run again (100,000 each). Students believe a number they watched appear

- **The model runs without Phaser.** `deno eval 'import { Deck } from "./src/Deck.ts"; const d = new Deck(); console.log(String(d.draw()), String(d.draw()));'` from the intermediate project folder. No browser, no Phaser - that is what model/view buys
- **Remove the state check** (session 1) - and then, in the advanced version, remove the `multiplier === 0` part of the check in `guess()` and try H on a king (nothing breaks; the guess is just always lost - why was it greyed out?)

- **Easing:** `Sine.easeIn` / `easeOut` versus `Linear` on the flip; `Back.easeOut` on the title's axes
 - **Turn the polish off** one piece at a time (session 2)
-

Discussion questions

- The simple, intermediate and advanced versions have the same odds. Why is the advanced one more fun? Which single feature adds the most?
 - A tie "does not count". What would change if a tie counted as a loss? As a win? Where in the code would you change it, and what else would need to change (text, the title screen)?
 - `Card` has no `x`, `y` or `faceUp`. Where do those live, and why? What would go wrong if `Card` extended `Phaser.GameObjects.Image`?
 - `winningRanks()` pretends all 13 ranks are equally likely. When is that badly wrong? (Late in the deck.) Should the multipliers use the real deck? Would that make the game better or just harder to understand?
 - Is the pot-and-cash-out design fair? What does a cautious player's score look like against a reckless one's? (Challenge 6 lets you test it.)
 - `GameOverScene` in the intermediate version handles both winning and losing. When should an ending be its own scene?
-

Extension ideas

- a "bet" before each guess: the player chooses how much of their banked score to risk
- sound pitch rising with the streak (`this.sound.play(key, { detune: inARow * 100 })`)
- a two-player version, taking turns, with the history row shared
- move `Card`, `Deck` and `Button` into a shared folder and use them in Chapter 13's memory game
- unit tests for `judge()`, `riskMultiplier()` and `Deck` with `Deno.test` - they need no browser

Assessment ideas

- Practical: "add a 'same' guess that pays x5 and wins only on a tie" - touches `Guess`, `judge()`, `riskMultiplier()`, a new button, and nothing in `Card` or `CardSprite`. Marking focuses on where the changes went
- Code reading: show a shuffle written with `sort(() => Math.random() - 0.5)` and ask what is wrong and how to show it is wrong
- Short answer: explain, with the flip as the example, why `reveal()` is called from the tween's `onComplete` rather than straight after `flipTo()`
- Design: "list three changes that would only touch the view, and three that would only touch the model"

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (`<!-- CHALLENGE n -->` in HTML).

1. Aces high

Project: `solutions/ch12_challenge_1_aces_high` (from `ch12_higher_or_lower_simple`)

Goal: see the point of `value` being separate from `rank`.

`src/Card.ts`

```
// what "higher" and "lower" compare.
// CHALLENGE 1: aces are high - an ace is worth 14, one more than a king.
// The rank stays 1, so the
// ace's picture (and toString) are unchanged: only the rules see the
// difference.
public get value(): number {
    return this.rank === 1 ? 14 : this.rank;
}
```

plus the rule text at the bottom of `GameScene` changed to "Aces are high".

One line of rules. `judge()` compares `value`, so it needs no change; `frameFor()` uses `rank`, so the ace still shows the ace picture. Tested with the harness: king face up, H, an ace drawn - "Ace of clubs - right!".

Look for: the change made in `value`, not `rank`. The most common wrong answer is to make an ace's **rank** 14 (in `random()` or the constructor). The rules then work, but the picture breaks: `frameFor()` gives `suit * 13 + 13`, the *next* suit's ace - and the ace of spades becomes frame 52, a card back. This is the model/view point of the chapter in miniature; worth showing the class.

2. Cards left

Project: `solutions/ch12_challenge_2_cards_left` (from `ch12_higher_or_lower_intermediate`)

Goal: add to a class's public interface without exposing its insides.

`src/Deck.ts`

```
// CHALLENGE 2: how many cards are still in the deck. A getter, so it reads like a field -  
// deck.size - but cannot be changed from outside.  
public get size(): number {  
    return this.cards.length;  
}
```

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: called after every draw()  
private showCardsLeft(): void {  
    this.cardsLeftText.setText(`Cards left: ${this.deck.size}`);  
}
```

`showCardsLeft()` is called straight after both `draw()` calls (the first card in `create()`, and in `guess()`). Tested: 51 at the start, 50 after one guess.

Look for: a getter or method on `Deck`, not `cards` made `public` - that would let the scene push and pop cards itself. A student who updates the text in `update()` every frame has a working answer; ask whether it needs to change 60 times a second.

3. Probability hint

Project: `solutions/ch12_challenge_3_probability_hint` (from `ch12_higher_or_lower_intermediate`)

Goal: ask the model a question by passing it a function.

`src/Deck.ts`

```
// CHALLENGE 3: how many of the cards left pass a test. The test is a
function that takes a card
// and says true or false - e.g. deck.countWhere((card) => card.value > 7)
public countWhere(test: (card: Card) => boolean): number {
    return this.cards.filter(test).length;
}
```

src/scenes/GameScene.ts

```
private showChances(): void {
    const current = this.currentCard.value;
    const higher = this.deck.countWhere((card) => card.value > current);
    const lower = this.deck.countWhere((card) => card.value < current);
    this.higherHint.setText(`${this.percent(higher)} chance`);
    this.lowerHint.setText(`${this.percent(lower)} chance`);
}

// CHALLENGE 3: e.g. 31 of 50 cards -> "62%"
private percent(count: number): string {
    return `${Math.round((count / this.deck.size) * 100)}%`;
}
```

`showChances()` is called at the end of `waitForGuess()`, and `guess()` clears both hints. Tested: king of diamonds face up, 51 cards left - "0% chance" and "94% chance" (48 of 51).

Look for:

- counting the *deck*, not the 13 ranks - the challenge says so, and it is what makes the hint interesting late in a game
- the two chances not adding up to 100%: the cards of the same rank are ties. A good discussion point
- `(card: Card) => boolean` as a parameter type - Java's `Predicate<Card>`. A student who adds `countHigher(value)` and `countLower(value)` to `Deck` has a fine answer; `countWhere` is more general
- hints hidden during the reveal (otherwise they show stale numbers for the old card)

4. Jokers wild

Project: `solutions/ch12_challenge_4_jokers_wild` (from `ch12_higher_or_lower_intermediate`)

Goal: extend the model and the view in step, and make a texture in code.

The model - `src/Card.ts`, `src/Deck.ts`, `src/rules.ts`:

```
// CHALLENGE 4: a third parameter with a default value, so every existing  
new Card(suit, rank)  
// still works and makes an ordinary card  
constructor(suit: Suit, rank: number, isJoker: boolean = false) {  
    ...  
// CHALLENGE 4: two jokers, shuffled in with the rest  
    this.cards.push(Card.joker(), Card.joker());  
    ...  
// CHALLENGE 4: jokers are wild - whatever you guessed, a joker makes it  
right  
    if (next.isJoker) {  
        return "right";  
    }  
}
```

The picture - `src/scenes/PreloadScene.ts` draws a cream card with a gold star using `Graphics`, and saves it:

```
graphics.generateTexture(JOKER_KEY, CARD_WIDTH, CARD_HEIGHT);  
graphics.destroy();
```

The view - `src/objects/CardSprite.ts`:

```
// CHALLENGE 4: a joker is a different texture, so setTexture (picture AND  
frame), not just setFrame  
public showFace(card: Card): void {  
    if (card.isJoker) {  
        this.setTexture(JOKER_KEY);  
    } else {  
        this.setTexture(CARDS_KEY, CardSprite.frameFor(card));  
    }  
}
```

and `showBack()` becomes `setTexture(CARDS_KEY, BACK_FRAME)`, or a sprite that once showed a joker could never show the back again.

The scene - `src/scenes/GameScene.ts`:

```
// CHALLENGE 4: a card that can be guessed against - jokers drawn here are put aside
private drawPlayable(): Card {
  let card = this.deck.draw();
  while (card.isJoker) {
    card = this.deck.draw();
  }
  return card;
}
```

used for the first card, and in `moveOn()` when the revealed card is a joker. Tested with a rigged deck: 2 face up, L, a joker - "Joker - right!", streak 1; the next card to beat is the 9 under it, and the next card turns back to the blue back (texture `cards`, frame 52).

Look for:

- the default parameter, so no existing `new Card(...)` call changed
- `setTexture` rather than `setFrame` - a student who writes `setFrame(JOKER_KEY)` gets the console warning `Texture "cards" has no frame "joker"`, and the sprite shows frame 0 (the ace of clubs) instead
- the joker handled in `judge()` (the model), not by special-casing the scene's messages
- the texture made after loading, in `create()` (or at the start of `GameScene`), with the `Graphics` destroyed afterwards
- a joker never becoming the card to beat - otherwise `judge()` compares against rank 0, and "lower" can never win

5. Double or nothing

Project: `solutions/ch12_challenge_5_double_or_nothing` (from `ch12_higher_or_lower_advanced`)

Goal: add a new state to the state machine, and a new action that uses the existing pieces.

src/Card.ts

```
// CHALLENGE 5: diamonds and hearts are the red suits
public get isRed(): boolean {
    return this.suit === "diamonds" || this.suit === "hearts";
}
```

src/scenes/GameScene.ts

```
// CHALLENGE 5: "gambling" - a double-or-nothing card is being dealt; no
// guesses or cash outs
type GameState = "dealing" | "waiting" | "revealing" | "gambling" | "over";
...
private doubleOrNothing(): void {
    if (this.state !== "waiting" || this.pot === 0) {
        return;
    }
    this.state = "gambling";
    this.setButtonsEnabled(false);
    this.messageText.setText("Double or nothing: red doubles the pot, black
loses it...");

    const card = this.deck.draw();
    this.cardsPlayed = this.cardsPlayed + 1;
    this.deckPile.setVisible(!this.deck.isEmpty());
    this.dealTo(this.nextSprite, NEXT_X, card, () => {
        this.settleGamble(card);
    });
}
```

`settleGamble(card)` doubles and banks the pot on red (with the cash-out coin sound and sparkles), or empties it on black (with the wrong sound and a shake), sets `inARow` to 0, and after a pause adds the card to the history row, hides the next-card sprite, and either ends the game (deck empty) or calls `waitForGuess()`. The card to beat does not change. D is added to the keys line, the controls in `index.html` and the README.

Tested with a rigged deck: D with an empty pot does nothing; H (pot 10), D, queen of hearts - score 20; H pressed during the gamble is ignored; H (pot 20), D, 3 of clubs - pot gone, score still 20, lives still 3.

Look for:

- a new state (not reusing `"revealing"`) and the existing `state !== "waiting"` checks doing the rest - H, L and C are all blocked during the gamble with no extra code
 - `isRed` in `Card` (the model) rather than checking suits in the scene
 - the deck-empty case handled - a gamble can draw the last card
 - reuse of `dealTo()`, `floatText()`, `pulse()` and the sparkles rather than copies
-
-

6. The computer plays the odds

Project: `solutions/ch12_challenge_6_computer_player` (from `ch12_higher_or_lower_advanced`)

Goal: drive the game from code through the same methods the player uses, and write a simple strategy.

`Deck` gains `countWhere()` (as in challenge 3) and a `size` getter. In `src/scenes/GameScene.ts`, the end of `waitForGuess()`:

```
// CHALLENGE 6: every time the game is ready for a move, the computer (if playing) makes one  
if (this.autoPlay) {  
    this.scheduleComputerMove();  
}
```

and the strategy:

```

private computerMove(): void {
  if (!this.autoPlay || this.state !== "waiting") {
    return; // switched off while thinking, or something else is happening
  }
  const current = this.currentCard.value;
  const higher = this.deck.countWhere((card) => card.value > current);
  const lower = this.deck.countWhere((card) => card.value < current);
  const chance = Math.max(higher, lower) / this.deck.size;

  if (this.pot > 0 && chance < CASH_OUT_BELOW) {
    this.cashOut(); // cashOut() calls waitForGuess(), which schedules the
    next move
    return;
  }
  // (only equal counts - e.g. both 0 when every card left is a tie - need
  the multiplier check,
  // so the computer never picks a guess the game will refuse, such as
  "higher" on a king)
  const guess: Guess = higher > lower || riskMultiplier("lower",
  this.currentCard) === 0 ? "higher" : "lower";
  this.guess(guess);
}

```

`scheduleComputerMove()` is a `this.time.delayedCall(THINK_TIME, ...)`. The A key toggles `autoPlay`, shows "COMPUTER PLAYING", and schedules a move straight away if the game is waiting. `autoPlay` is reset in `init()`, so each game starts with the player in charge.

Tested by switching the computer on and letting it play a whole game with the scene's clocks sped up (`time.timeScale` and `tweens.timeScale` set to 4 from the harness): it played 36 cards on its own, cashed out several times, and finished "Out of lives!" with 990 points and a longest run of 10.

Look for:

- the computer calling `guess()` and `cashOut()` - the same methods as the keys - so it cannot cheat or skip the state checks
- the check at the top of `computerMove()` : without it, pressing A twice quickly schedules two moves, or the computer moves after being switched off
- no `update()` polling ("is it waiting now? now?") - the event-driven version is simpler

- a guess the game would refuse (`riskMultiplier` 0) would leave the computer stuck waiting for a move that never happens - ask students how they would spot that bug
- extension: run many games and print the average score for different `CASH_OUT_BELOW` values - a real experiment in strategy

Slides

Chapter 12

Higher or lower

One game, three versions



1 / 20

Today

- what makes a guessing game fun
- a `Card` , a `Deck` , the rules - **no Phaser**
- a `CardSprite` and a `Button` - **the view**
- a proper shuffle (and a broken one)
- game state: ignoring the player at the right moments
- a flip made of two tweens
- risk, reward, and cashing out

2 / 20

What makes it fun?

The luck is the same in all three versions. So why is one more fun?

- **risk** - "higher" on a 2 is safe; on a jack it is a gamble. Show it; pay for it
- **streaks** - five in a row feels like skill. Let the player lose it
- **the reveal** - the half-second while the card turns *is* the game

Rules: aces low (1); a tie does not count

3 / 20

Try it: simple

- `ch12_higher_or_lower_simple`
- H = higher, L = lower
- five right in a row wins
- wrong: back to zero
- one scene, one small class



4 / 20

A card is data

```
export type Suit = "clubs" | "diamonds" | "hearts" | "spades";

export class Card {
  public readonly suit: Suit;
  public readonly rank: number; // 1 = ace, 2 - 10, 11 = jack, 12 = queen, 13 = king
  ...
  public get value(): number {
    return this.rank;
  }
}
```

- union of string literals: only those four
- `readonly` = Java's `final` ; `get` = read like a field
- no picture, no position, no Phaser

5 / 20

Which picture?

Frames of cards.png: $\text{frame} = \text{suit} * 13 + (\text{rank} - 1)$

	A (1)	2 (2)	3 (3)	4 (4)	5 (5)	6 (6)	7 (7)	8 (8)	9 (9)	10 (10)	J (11)	Q (12)	K (13)
0 clubs	0	1	2	3	4	5	6	7	8	9	10	11	12
1 diamonds	13	14	15	16	17	18	19	20	21	22	23	24	25
2 hearts	26	27	28	29	30	31	32	33	34	35	36	37	38
3 spades	39	40	41	42	43	44	45	46	47	48	49	50	51
	52	53	52 and 53: the two backs										

Queen of hearts: suit 2, rank 12, so $\text{frame } 2 * 13 + (12 - 1) = 37$

```
return SUITS.indexOf(card.suit) * 13 + (card.rank - 1);
```

6 / 20

The rules - and when to listen

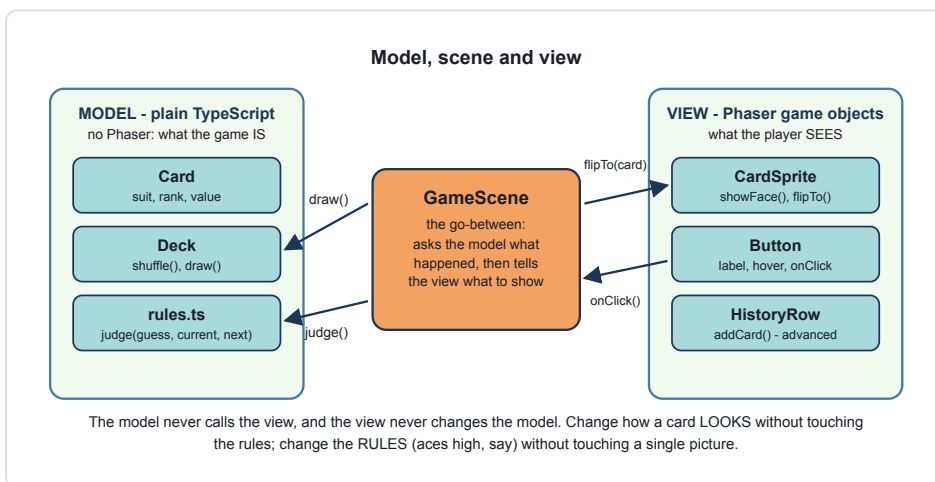
```
type Outcome = "right" | "wrong" | "tie";
type GameState = "waiting" | "revealing" | "won";

private guess(guess: Guess): void {
  if (this.state !== "waiting") {
    return; // a card is still being revealed, or the game is over
  }
  this.state = "revealing";
  ...
  const outcome = this.judge(guess, this.currentCard, next);
}
```

`judge()` only looks at cards. The scene decides what each outcome *looks* like.

7 / 20

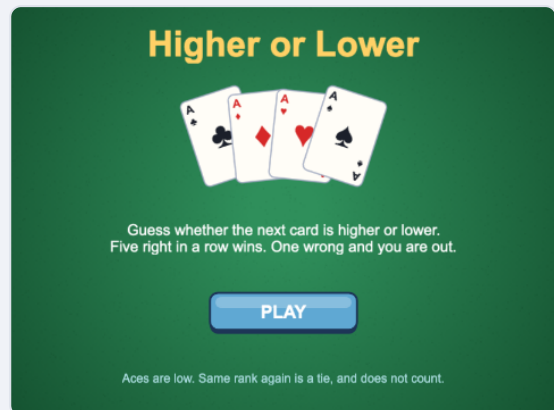
Model and view



8 / 20

Try it: intermediate

- `ch12_higher_or_lower_intermediate`
- a shuffled deck: no repeats
- cards flip; buttons; five lights
- one wrong and you are out
- find: `Card.ts`, `Deck.ts`, `rules.ts` - which import Phaser?



9 / 20

Fisher-Yates

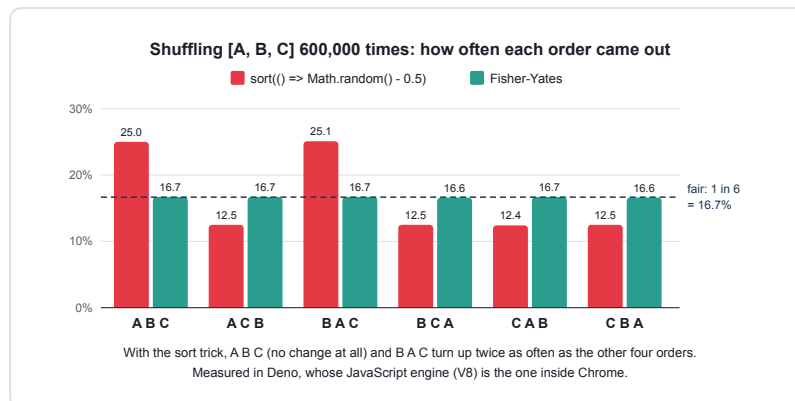
```
public shuffle(): void {
  for (let i = this.cards.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1)); // 0 to i, inclusive
    const swap = this.cards[i];
    this.cards[i] = this.cards[j];
    this.cards[j] = swap;
  }
}
```

- fill each place, from the back, with a fair pick of what is left
- one pass; every order equally likely
- `Phaser.Utls.Array.Shuffle` does the same

10 / 20

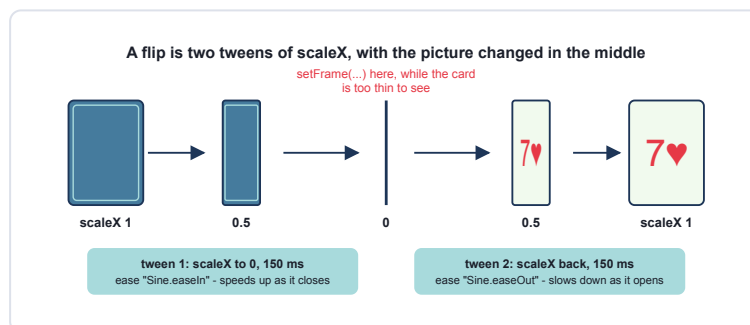
The shuffle you will find online

```
cards.sort(() => Math.random() - 0.5); // DON'T
```



11 / 20

A flip is two tweens



```
this.scene.tweens.chain({
  targets: this,
  tweens: [
    {
      scaleX: 0,
      duration: FLIP_TIME / 2,
      ease: "Sine.easeIn",
      onComplete: () => {
        ...
      },
    },
    { ... },
  ],
  onComplete: onComplete,
});
```

The first tween's `onComplete` calls `showFace(card)` (or `showBack()`) while the card is edge-on

12 / 20

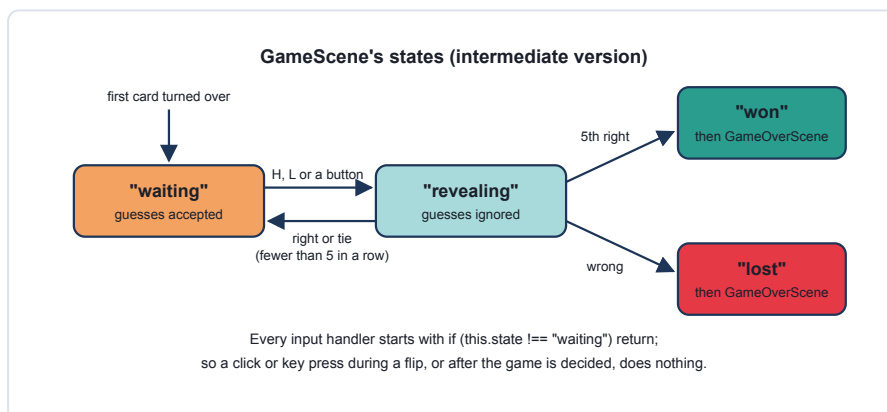
A button

```
export class Button extends Phaser.GameObjects.Container {  
  ...  
  this.add([this.background, this.label]);  
  this.background.setInteractive({ useHandCursor: true });  
  
  this.background.on("pointerover", () => {  
    this.background.setTexture(BUTTON_OVER_KEY);  
  });  
  ...  
  this.background.on("pointerup", () => {  
    ...  
    onClick();  
  });  
}
```

- a **Container**: one object holding a picture and a label
- normal / over / down pictures; act on **pointerup**
- the caller passes in what to do

13 / 20

The state machine



`reveal()` runs from the flip's `onComplete` : see the card **first**, then hear the result

14 / 20

Try it: advanced

- `ch12_higher_or_lower_advanced`
- one deck, three lives
- a **pot**: more for risk, more for a long run
- **cash out** (C) to bank it
- wrong: lose the pot **and** a life



15 / 20

Paying for risk

```
export function winningRanks(guess: Guess, current: Card): number {  
  if (guess === "higher") {  
    return 13 - current.value;  
  }  
  return current.value - 1;  
}  
  
export function pointsFor(multiplier: number, inARow: number): number {  
  return BASE_POINTS * multiplier * inARow;  
}
```

- `riskMultiplier()` : 7+ winning ranks x1, 4-6 x2, 1-3 x3, 0 = impossible
- the buttons show it: `HIGHER x1` , `LOWER x3`

16 / 20

The pot and cash out

- `score` - banked, safe
- `pot` - at risk
- only a score: always guess the likely way. No decisions
- with a pot: **when do you stop?**

```
this.score = this.score + this.pot;  
this.pot = 0;  
this.inARow = 0;
```



17 / 20

Dealing, history, records, polish

```
const oldCurrent = this.currentSprite;  
this.currentSprite = this.nextSprite;  
this.nextSprite = oldCurrent;
```

- two sprites swap jobs - they are references, as in Java
- `HistoryRow` - a `Container` : tell every card where it belongs
- `Records` - best score and longest run in local storage
- floating "+30", pulses, sparkles, a shake, a heart that pops

18 / 20

Summary

- risk, streaks, the reveal
- **model** (`Card` , `Deck` , `rules.ts`) apart from **view** (`CardSprite` , `Button`)
- unions of strings for suits, guesses, outcomes, states
- Fisher-Yates - not `sort(random)`
- a `state` checked at the top of every input
- flip = two tweens of `scaleX` , frame changed in between
- a pot you can lose makes the score interesting

19 / 20

Challenges

1. **Aces high** - an ace beats a king
2. **Cards left** - count down the deck
3. **Probability hint** - the real chance, from the deck
4. **Jokers wild** - two jokers, drawn in code
5. **Double or nothing** - red doubles, black loses
6. **The computer plays the odds** - press A and watch

Next: **Chapter 13 - Memory match**

20 / 20